

Programming Simplified with GNU Scientific Library

Welcome back! I do hope that my previous article was convincing enough for you to really try out GSL. I had covered installation and some basic programs in that article. In this second article in the series, I will tell you how to go about creating portable programs, and will also discuss GSL errors.



GSL documentation is very well organised, easy to access and simple. There are three basic sources of GSL documentation. They are all practically similar, so you could choose any one of them:

- M. Galassi et al, GNU Scientific Library Reference Manual (Edition 1.14, for GSL Version 1.14), ISBN 0954612078.
- The *info* pages, accessible via the command *info gsl-ref*, were installed along with GSL.
- Online documentation is also available at the GSL homepage.

These documentation sources contain a lot of sample programs in almost all the routines. You can go through them to get a brief idea about writing programs.

Writing portable programs

GSL really seems to get a positive rating in all respects, but one major issue is code portability. You can either ensure that the other machine has GSL installed on it, or write portable code. GSL actually lets you do this, provided you agree to the terms of the GPL.

If you have read my previous article, I had suggested that you install the package from source code. The reason for that will now be clear. The whole package is divided into folders that contain the routines that we are going to use.

There are two ways to make portable programs: one is to take all the header files and the corresponding '.c' files, and include them locally; the other is to copy the GSL function definitions (and supporting definitions) into your program's source code. The first may be easier, and there's the advantage that if you wish to modify some of GSL for your program's specific needs, you can do it easily. In the second case, finding the functions and the respective data types could be a tedious job.

Most of the routines are independent as such, and don't have common or shared files except error files. The latter are shared by almost all the routines available, and I will cover their significance in this article.